

Test 4 :

Q1. Duplicate.java file Nested loop is slower version $O(n^2)$
compare with is fast cause $O(n)$

Q2. In concatenate file string, immutable new copy
 $O(n^2)$ slower version compare to
string builder $O(n)$ faster version

Q3. ListTraps: ArrayList.get(i) takes $O(n^2)$ slower version
& deque.get(i) $O(n)$ faster.

Dry Run

1) Demo.java

{ [()] }

i	ch	Stack	top
0		{ }	0
1	[	{ [	1
2	(	{ [(	2
3	)	{ [	1
4	]	{ }	0
5	}	Empty	-1

2) StackArrayInt.java.

step	operations	Stack	top
1	Capacity = 4	Empty	-1
2	push 10	10	0
3	push 20	10 20	1
4	push 30	10 20 30	2
5	push 40	10 20 30 40	3
6	Display	40 30 20 10	3
7	Peek	10 20 30 40	3
8	pop 40	10 20 30	2
9	Display	30 20 10	2
10	Exit	30 20 10	2

3) Reverse String Stack Demo.java

steps	operation	stack	Reversed
1	push c	c	" "
2	push A	c A	" "
3	push T	c A T	" "
4	pop T	c A	T
5	pop A	c	T A
6	pop c	Empty	<u>T A C</u> ← Reversed

4) Parenthesis Match Demo.java

a) $(x(x+y))$

step	operation	stack	Result
1	push (	(	Push
2	x	(	ignore
3	(	((	push
4	x	((	ignore
5	+	((	ignore
6	y	((	ignore
7	pop)	(	(is matching
8	pop)	empty	(is matching

→ Balanced

b) $(a+b) * (c)$

step	operation	stack	Result
1	push (	(	Push
2	a	(	ignore
3	+	(	ignore
4	b	(	ignore
5	)	(	is not matching

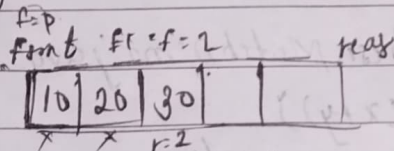
→ Not Balanced

Expression Evaluation Demo.java

Step	Operation	Stack	Result
1	1	1	push 1
2	2	1 2	
3	3	1 2 3	
4	*	1 2 3 3	
5	+	1 2 6	3+3=6
6	5	1 2 6 5	push 5

result = 5 (5 will pop)

8) QueDemo.java



i	operation	Queue	front / rear
1	Initial	Empty	-1 / -1
2	Enque 10	10	0 / 0
3	Enque 20	10 20	0 / 1
4	Enque 30	10 20 30	0 / 2
5	Display	10 20 30	0 / 2
6	Deque 10	20 30	1 / 2
7	Deque 20	30	2 / 2
8	Exit	30	2 / 2

9) CircularQueue.java

i	operation	Queue (index 0-4)	front / rear
1)	Initial	_, _, _, _, _	-1 -1
2)	Enque 10	10, _, _, _, _	0 / 0
3)	Enque 20	10, 20, _, _, _	0 / 1
4)	Enque 30	10, 20, 30, _, _	0 / 2
5)	Enque 40	10, 20, 30, 40, _	0 / 3
6)	Deque 10	_, 20, 30, 40, _	1 / 3
7)	Deque 20	_, _, 30, 40, _	2 / 3
8)	Enque 50	50, _, 30, 40, 50	2 / 4
9)	Enque 60	50, _, 30, 40, 50	2 / 0
10)	Display	30, 40, 50, 60	2 / 0